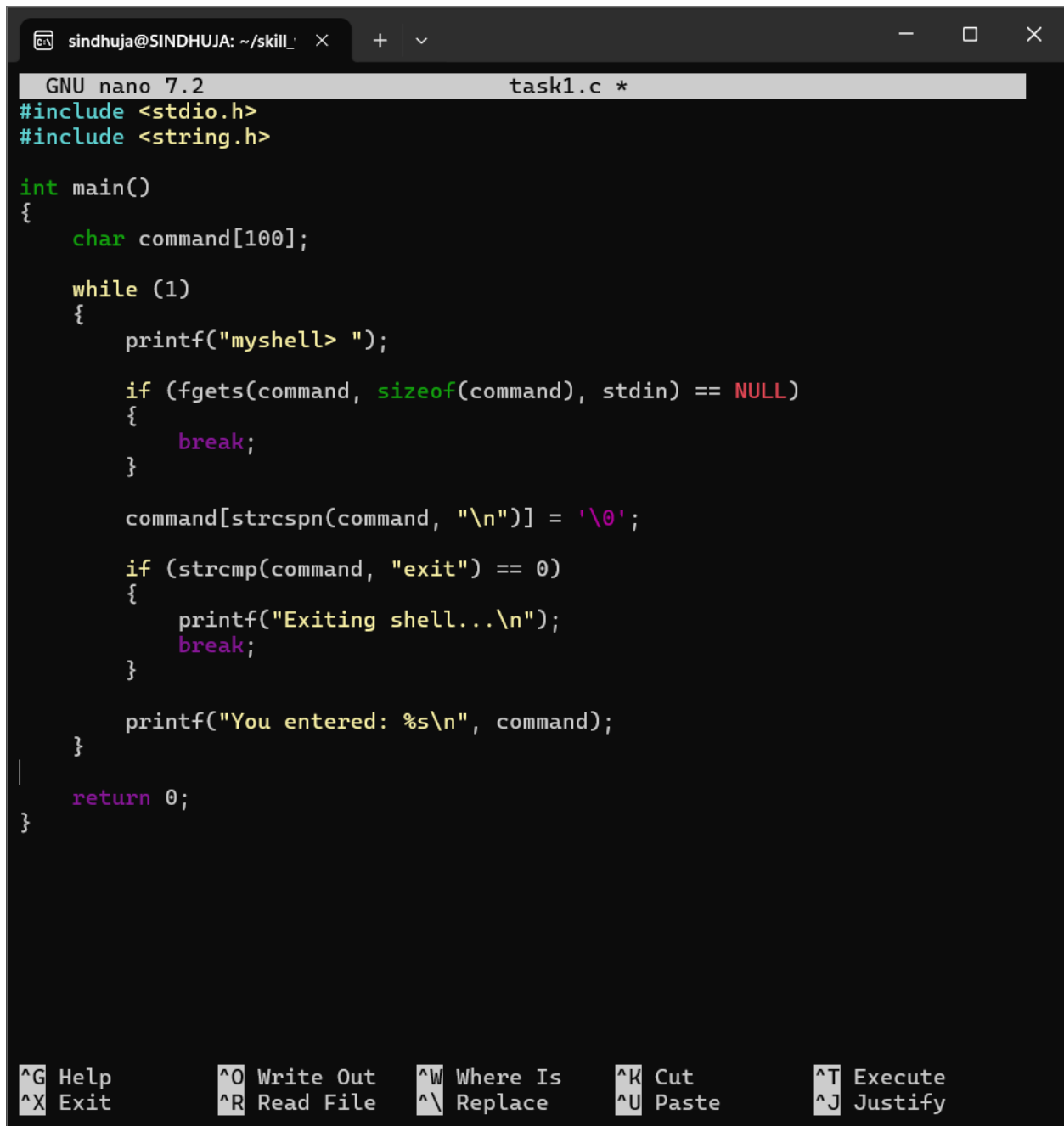


Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop



```
GNU nano 7.2 task1.c *
#include <stdio.h>
#include <string.h>

int main()
{
    char command[100];

    while (1)
    {
        printf("myshell> ");

        if (fgets(command, sizeof(command), stdin) == NULL)
        {
            break;
        }

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        printf("You entered: %s\n", command);
    }

    return 0;
}
```

Help **Write Out** **Where Is** **Cut** **Execute**
Exit **Read File** **Replace** **Paste** **Justify**

```
sindhuja@SINDHUJA: ~/skill_ x + v
sindhuja@SINDHUJA:~$ mkdir skill_week3
sindhuja@SINDHUJA:~$ cd skill_week3
sindhuja@SINDHUJA:~/skill_week3$ pwd
/home/sindhuja/skill_week3
sindhuja@SINDHUJA:~/skill_week3$ nano task1.c
sindhuja@SINDHUJA:~/skill_week3$ gcc task1.c -o task1
sindhuja@SINDHUJA:~/skill_week3$ ./task1
myshell> hello
You entered: hello
myshell> Hello world
You entered: Hello world
myshell> Operating System
You entered: Operating System
myshell> exit
Exiting shell...
sindhuja@SINDHUJA:~/skill_week3$ |
```

Task 2: Using the following system calls, copy the content from File1.txt to File2.txt.

1. **Open()**
2. **Read()**
3. **Write()**
4. **Close()**

open()

syntax: `open(" filename.txt", modes, permissions)`

Example: `open("First_Prgm.txt", O_CREAT | O_WRONLY, 0644);`

Table 1: Modes of Open system call

Flag	Purpose
O_RDONLY	Open for read only
O_WRONLY	Open for write only
O_RDWR	Open for both read and write
O_CREAT	Create file if it does not exist
O_TRUNC	Truncate (empty) an existing file

Flag	Purpose
O_APPEND	Append data to the end of the file
O_EXCL	Fail if file already exists (used with O_CREAT)

read()

Syntax: read(int fd, void *buf, size_t count);

Example: read(fd, buffer, 100)

#include <stdio.h>

#include <unistd.h>

```
int main()
{
    char buffer[100];
    int n;
    printf("Enter some text: ");
    n = read(0, buffer, sizeof(buffer) - 1);
    buffer[n] = '\0'; // Null-terminate the string
    printf("You entered: %s", buffer);
    return 0;
}
```

write()

Syntax: write(int fd, const void *buf, size_t count);

Example: write(fd, message, 11);

#include <unistd.h>

```
int main()
{
    write(1, "Hello World\n", 12);
    return 0;
}
```

```
sindhuja@SINDHUJA:~/skill_week3$ nano File1.txt
sindhuja@SINDHUJA:~/skill_week3$ cat File1.txt
Hello World
Welcome to Linux System Programming.
This is Task 2.
```

```
sindhuja@SINDHUJA: ~/skill_ × + ∨
GNU nano 7.2 File1.txt
Hello World
Welcome to Linux System Programming.
This is Task 2.
```

```
sindhuja@SINDHUJA: ~/skill_ × + ∨
sindhuja@SINDHUJA:~/skill_week3$ nano task2.c
sindhuja@SINDHUJA:~/skill_week3$ cat task2.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int source, destination;
    char buffer[100];
    int bytesRead;

    source = open("File1.txt", O_RDONLY);

    if (source == -1)
    {
        printf("Error opening File1.txt\n");
        return 1;
    }

    destination = open("File2.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination == -1)
    {
        printf("Error creating File2.txt\n");
        close(source);
        return 1;
    }

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
    {
        write(destination, buffer, bytesRead);
    }

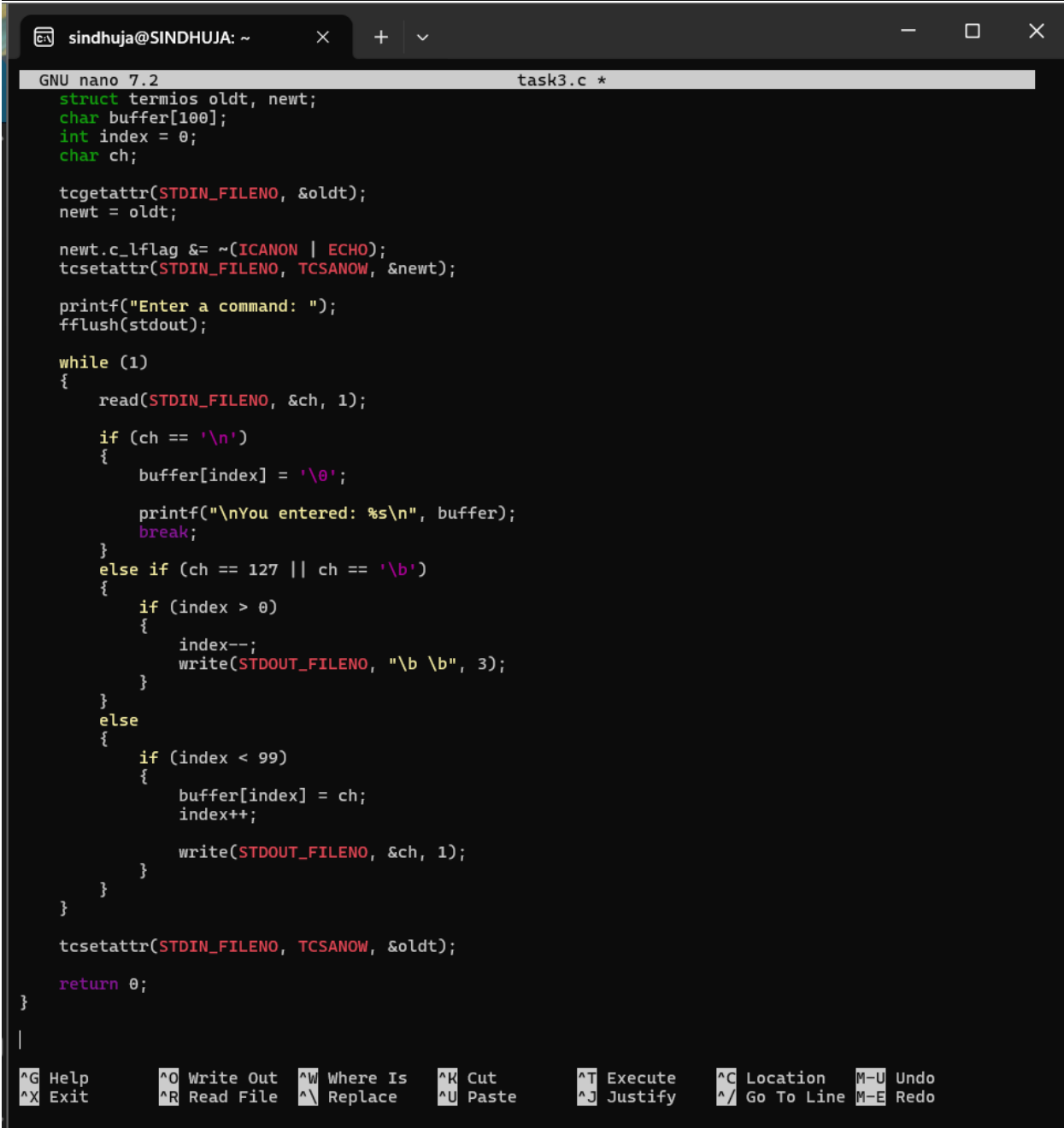
    close(source);
    close(destination);

    printf("Contents copied successfully.\n");

    return 0;
}
sindhuja@SINDHUJA:~/skill_week3$ gcc task2.c -o task2
sindhuja@SINDHUJA:~/skill_week3$ ./task2
Contents copied successfully.
sindhuja@SINDHUJA:~/skill_week3$ cat File2.txt
Hello World
Welcome to Linux System Programming.
This is Task 2.
```

Task 3: To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

```
sindhuja@SINDHUJA:~$ nano task3.c
sindhuja@SINDHUJA:~$ gcc task3.c -o task3
sindhuja@SINDHUJA:~$ ./task3
Enter a command: Hello
You entered: Hello
sindhuja@SINDHUJA:~$ |
```



```
GNU nano 7.2 task3.c *
struct termios oldt, newt;
char buffer[100];
int index = 0;
char ch;

tcgetattr(STDIN_FILENO, &oldt);
newt = oldt;

newt.c_lflag &= ~(ICANON | ECHO);
tcsetattr(STDIN_FILENO, TCSANOW, &newt);

printf("Enter a command: ");
fflush(stdout);

while (1)
{
    read(STDIN_FILENO, &ch, 1);

    if (ch == '\n')
    {
        buffer[index] = '\0';

        printf("\nYou entered: %s\n", buffer);
        break;
    }
    else if (ch == 127 || ch == '\b')
    {
        if (index > 0)
        {
            index--;
            write(STDOUT_FILENO, "\b \b", 3);
        }
    }
    else
    {
        if (index < 99)
        {
            buffer[index] = ch;
            index++;

            write(STDOUT_FILENO, &ch, 1);
        }
    }
}

tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

return 0;
}
```

Terminal Window: sindhuja@SINDHUJA: ~

Editor Title Bar: GNU nano 7.2 task3.c *

Footer:

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

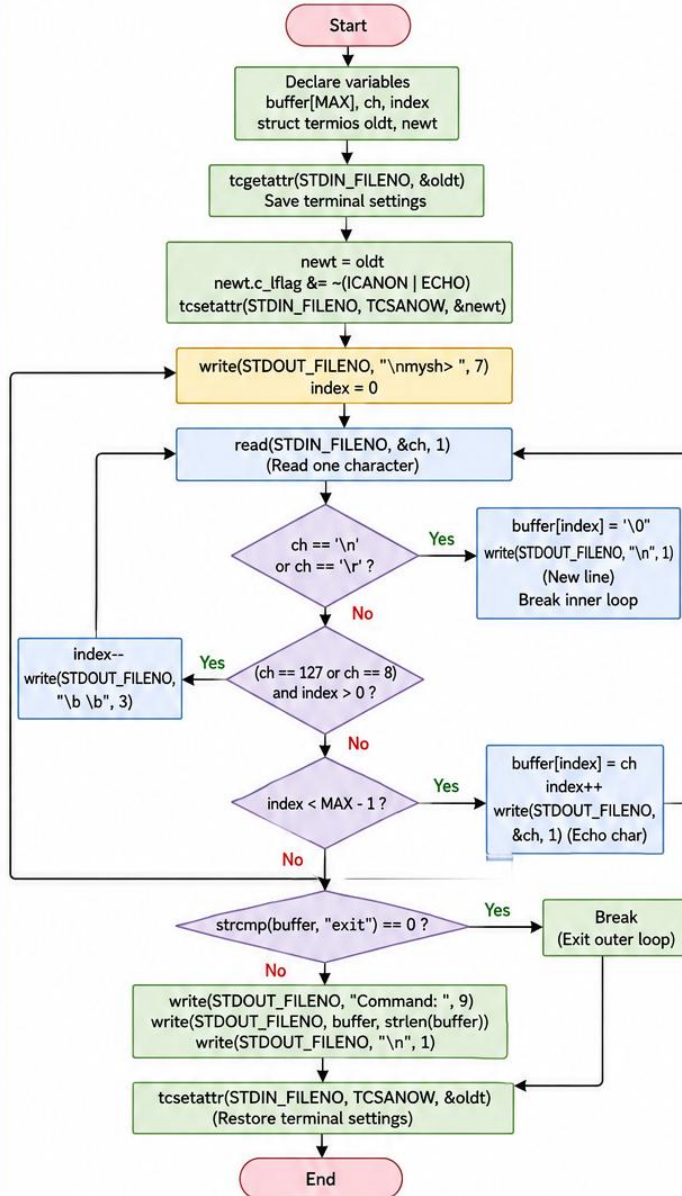
ALGORITHM & FLOWCHART

Keyboard Input Handling Using System Calls (read, write, termios)

ALGORITHM

1. Start.
2. Declare variables:
 - buffer[MAX]
 - char ch
 - int index
 - struct termios oldt, newt
3. Save the current terminal settings using `tcgetattr(STDIN_FILENO, &oldt)`.
4. Copy the terminal settings:
`newt = oldt`
5. Disable canonical mode and echo:
`newt.c_lflag &= ~(ICANON | ECHO)`
6. Apply the new terminal settings using `tcsetattr(STDIN_FILENO, TCSANOW, &newt)`.
7. Repeat forever
 - (a) Display the prompt using `write()`:
"mysh> "
 - (b) Set `index = 0`.
 - (c) Read characters one by one using `read()`.
 - (d) If the character is Enter ('\n' or '\r'):
 - `buffer[index] = '\0'`
 - Print new line.
 - Exit the inner loop.
 - (e) Else if the character is Backspace (ASCII 8 or 127) and `index > 0`:
 - `index--`
 - Print "\b \b" to remove last character.
 - (f) Else if `index < MAX - 1`:
 - Store the character in buffer.
 - `index++`
 - Echo the character using `write()`.
 - (g) After exiting the inner loop, compare the command in buffer.
 - If command is "exit", break the outer loop.
 - Otherwise, print "Command: " followed by the command.
8. Restore the original terminal settings using `tcsetattr(STDIN_FILENO, TCSANOW, &oldt)`.
9. End.

FLOWCHART



SYSTEM CALLS / FUNCTIONS USED

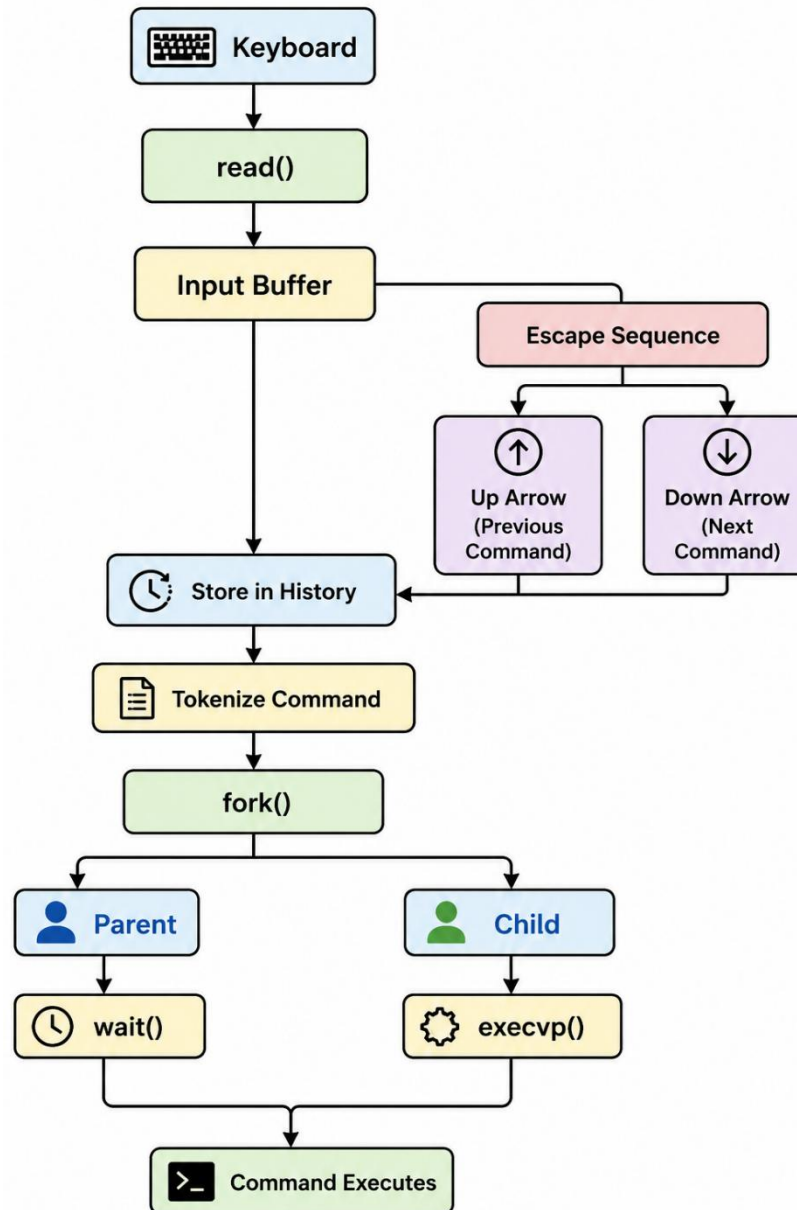
Function / System Call	Purpose
<code>read()</code>	Reads one character from keyboard (stdin).
<code>write()</code>	Writes output to terminal (stdout).
<code>tcgetattr()</code>	Gets current terminal attributes.
<code>tcsetattr()</code>	Sets terminal attributes (raw / canonical mode).
<code>strcmp()</code>	Compares strings (used to check "exit").

TERMINAL MODE

- Canonical mode (default): Input is line buffered. Backspace is handled by terminal driver.
- Non-canonical (raw) mode: Input is available character by character; program handles Backspace, Enter, etc.
- We disable ICANON (canonical) and ECHO to read characters immediately and handle them in the program.

This program uses system calls (read, write) and terminal control (termios) to capture keyboard input, handle backspace, process enter key, manage input buffer and support multi-character commands.

Task 4: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality





sindhuja@SINDHUJA: ~



GNU nano 7.2

```
#include <stdio.h>
#include <unistd.h>
#include <termios.h>
#include <string.h>

#define MAX_COMMANDS 10
#define MAX_LENGTH 100

int main()
{
    struct termios oldt, newt;
    char history[MAX_COMMANDS][MAX_LENGTH];
    char buffer[MAX_LENGTH];
    char ch;
    int index = 0;
    int history_count = 0;
    int history_position = 0;

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;

    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    while (1)
    {
        index = 0;
        history_position = history_count;

        printf("\nEnter a command: ");
        fflush(stdout);

        while (1)
        {
            read(STDIN_FILENO, &ch, 1);

            if (ch == '\n')
            {
                buffer[index] = '\0';

                if (strcmp(buffer, "exit") == 0)
                {
                    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
                    printf("\nExiting...\n");
                    return 0;
                }

                if (index > 0)
                {
                    strcpy(history[history_count % MAX_COMMANDS], buffer);

                    if (history_count < MAX_COMMANDS)
                        history_count++;
                }

                printf("\nYou entered: %s\n", buffer);
                break;
            }

            else if (ch == 127 || ch == '\b')
            {
                if (index > 0)
                {
                    index--;
                    buffer[index] = '\0';
                }
            }
        }
    }
}
```

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Loca
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify	^/_ Go To

```

else if (ch == 127 || ch == '\b')
{
    if (index > 0)
    {
        index--;
        write(STDOUT_FILENO, "\b \b", 3);
    }
}

else if (ch == 27)
{
    char seq[2];

    read(STDIN_FILENO, &seq[0], 1);
    read(STDIN_FILENO, &seq[1], 1);

    if (seq[0] == '[' && seq[1] == 'A')
    {
        if (history_count > 0 && history_position > 0)
        {
            history_position--;

            while (index > 0)
            {
                write(STDOUT_FILENO, "\b \b", 3);
                index--;
            }

            strcpy(buffer, history[history_position]);

            index = strlen(buffer);
            write(STDOUT_FILENO, buffer, index);
        }
    }
}

```

```

else if (seq[0] == '[' && seq[1] == 'B')
{
    if (history_position < history_count - 1)
    {
        history_position++;

        while (index > 0)
        {
            write(STDOUT_FILENO, "\b \b", 3);
            index--;
        }

        strcpy(buffer, history[history_position]);

        index = strlen(buffer);
        write(STDOUT_FILENO, buffer, index);
    }
}

else
{
    if (index < MAX_LENGTH - 1)
    {
        buffer[index++] = ch;
        write(STDOUT_FILENO, &ch, 1);
    }
}
}

tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

return 0;
}
|

```

```

sindhuja@SINDHUJA:~$ nano task4.c
sindhuja@SINDHUJA:~$ gcc task4.c -o task4
sindhuja@SINDHUJA:~$ ./task4

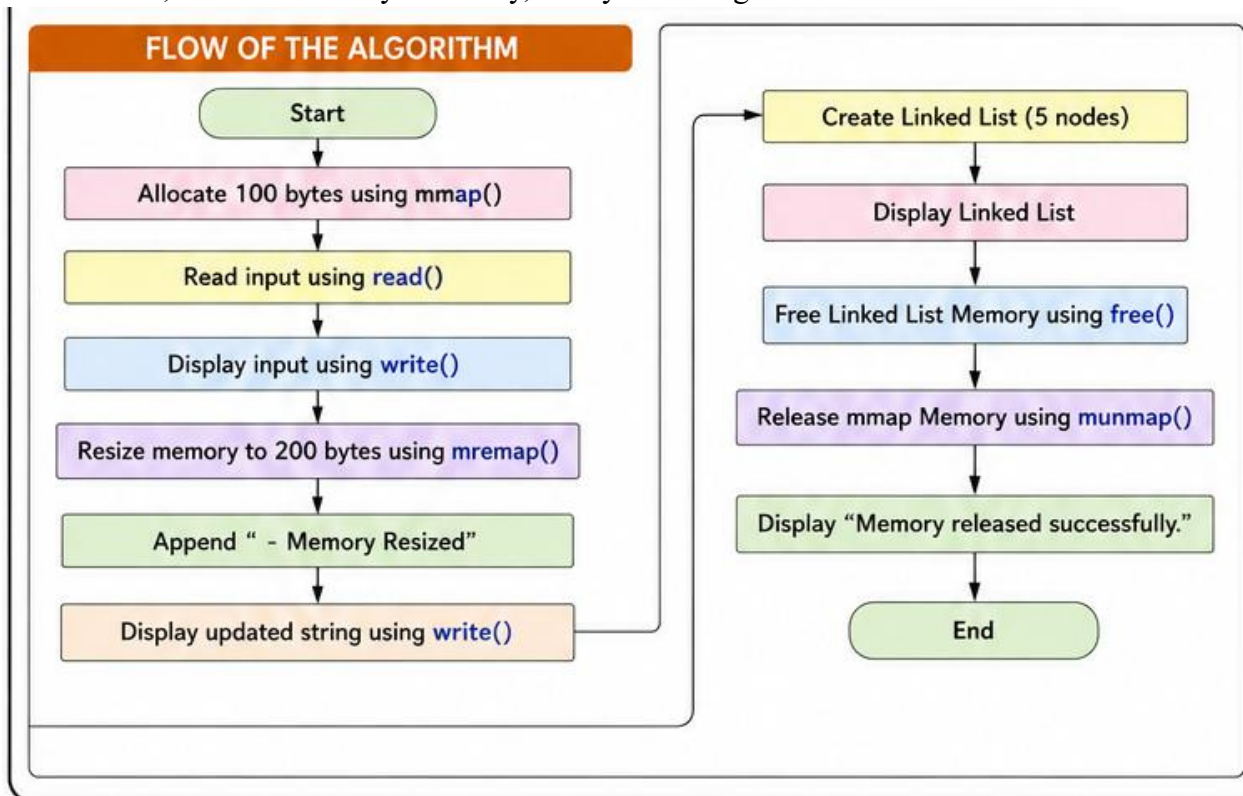
Enter a command: HELLO WORLD
You entered: HELLO WORLD

Enter a command: Operating System
You entered: Operating System

Enter a command: exit
Exiting...
sindhuja@SINDHUJA:~$ |

```

Task 5: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.



Validate the program with valgrind tool



sindhuja@SINDHUJA: ~



GNU nano 7.2

```
#define _GNU_SOURCE

#include <stdio.h>
#include <unistd.h>
#include <sys/mman.h>
#include <string.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

int main()
{
    char *buffer;
    int bytesRead;

    // Allocate 100 bytes using mmap()
    buffer = mmap(NULL, 100,
                  PROT_READ | PROT_WRITE,
                  MAP_PRIVATE | MAP_ANONYMOUS,
                  -1, 0);

    if (buffer == MAP_FAILED)
    {
        perror("mmap");
        return 1;
    }

    printf("Enter some text: ");
    fflush(stdout);

    // Read input using read()
    bytesRead = read(STDIN_FILENO, buffer, 99);

    if (bytesRead < 0)
    {
        perror("read");
        munmap(buffer, 100);
        return 1;
    }

    buffer[bytesRead] = '\0';

    // Display input
    printf("You entered: %s", buffer);

    // Resize memory from 100 to 200 bytes
    buffer = mremap(buffer, 100, 200, MREMAP_MAYMOVE);

    if (buffer == MAP_FAILED)
    {
        perror("mremap");
        return 1;
    }

    // Append text after resizing
    strcat(buffer, " - Memory Resized");
}
```

```

// Append text after resizing
strcat(buffer, " - Memory Resized");

printf("\nUpdated string: %s\n", buffer);

// Create linked list with 5 nodes
struct Node *head = NULL;
struct Node *temp;

for (int i = 1; i <= 5; i++)
{
    struct Node *newNode = malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    newNode->data = i;
    newNode->next = head;
    head = newNode;
}

// Display linked list
printf("\nLinked List: ");

temp = head;

while (temp != NULL)
{
    printf("%d -> ", temp->data);
    temp = temp->next;
}

printf("NULL\n");

// Free linked list memory
temp = head;

while (temp != NULL)
{
    struct Node *nextNode = temp->next;
    free(temp);
    temp = nextNode;
}

// Release mmap memory
munmap(buffer, 200);

printf("Memory released successfully.\n");

return 0;
}
|

```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^_ Replace

^K Cut
^U Paste

^T Execut
^J Justif

```
sindhuja@SINDHUJA:~/skill_week3$ nano task5.c
sindhuja@SINDHUJA:~/skill_week3$ gcc task5.c -o task5
sindhuja@SINDHUJA:~/skill_week3$ gcc task5.c -o task5
sindhuja@SINDHUJA:~/skill_week3$ ./task5
Enter some text: Hello World
You entered: Hello World

Updated string: Hello World
- Memory Resized

Linked List: 5 -> 4 -> 3 -> 2 -> 1 -> NULL
Memory released successfully.
sindhuja@SINDHUJA:~/skill_week3$ valgrind --version
valgrind-3.22.0
sindhuja@SINDHUJA:~/skill_week3$ valgrind --leak-check=full ./task5
==924== Memcheck, a memory error detector
==924== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==924== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==924== Command: ./task5
==924==
Enter some text: Hello World
You entered: Hello World

Updated string: Hello World
- Memory Resized

Linked List: 5 -> 4 -> 3 -> 2 -> 1 -> NULL
Memory released successfully.
==924==
==924== HEAP SUMMARY:
==924==    in use at exit: 0 bytes in 0 blocks
==924==    total heap usage: 6 allocs, 6 frees, 1,104 bytes allocated
==924==
==924== All heap blocks were freed -- no leaks are possible
==924==
==924== For lists of detected and suppressed errors, rerun with: -s
==924== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```